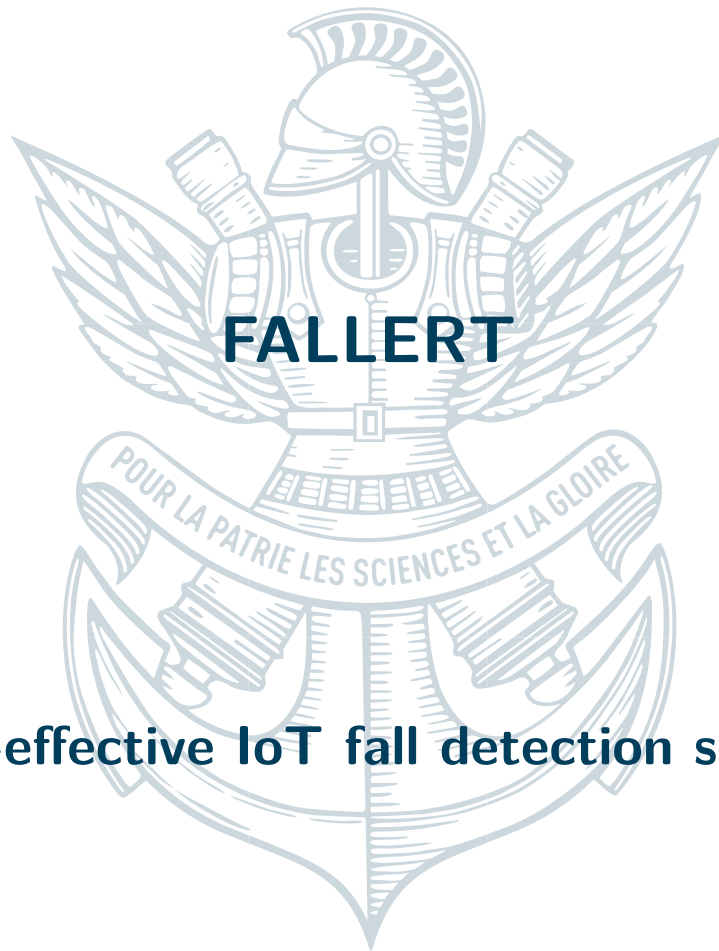




Cost-effective IoT fall detection system

March 16, 2025

Shalva BUTKHUZI, Zehua HUANG



1

INTRODUCTION

This project aims to develop a cost-effective fall detection system that can automatically notify family members of an injured individual in various scenarios. The system is specifically designed for elderly individuals, who are at a higher risk of fall-related injuries. According to the World Health Organization (WHO), falls are the leading cause of injury-related mortality worldwide, with the highest incidence observed among older adults in low- and middle-income countries [1]. An automated alert mechanism can facilitate timely intervention by family members, who can subsequently contact emergency medical services. Studies indicate that rapid medical response can mitigate the impact of falls, improve patient outcomes, and reduce the overall burden on healthcare systems [2].

Existing solutions, such as the Apple Watch, which introduced fall detection in 2020, provide an automated system that detects falls, triggers an alarm, and places emergency calls [3]. However, this device has certain limitations, including a maximum battery life of 36 hours in standard mode [4] and a starting price of approximately \$250, making it less accessible to people in low- and middle-income regions.

The proposed system offers several advantages over existing solutions. It is a more cost-effective alternative, increasing accessibility for a broader population. Additionally, it is designed with optimized battery efficiency, allowing for prolonged operation without frequent recharging, which is particularly beneficial for elderly users. Furthermore, the device is lightweight and designed for wearability, ensuring ease of use and comfort in daily activities. These factors contribute to a more practical and efficient solution for fall detection and emergency response.

2

SYSTEM DESIGN

2.1 SYSTEM DESIGN

The device employs two communication methods for transmitting alert messages: one through Wi-Fi and the other via a Bluetooth connection to a mobile phone (Figure 1). The mobile phone, in turn, can relay the alert signal using either the MQTT publish-subscribe messaging protocol or, as a backup, through the cellular network to send SMS messages. The MQTT protocol is preferred as the primary communication method due to its ability to support extensive alert information exchange between publishers and subscribers, its enhanced security compared to restricted two-way automated SMS, and its lower cost.

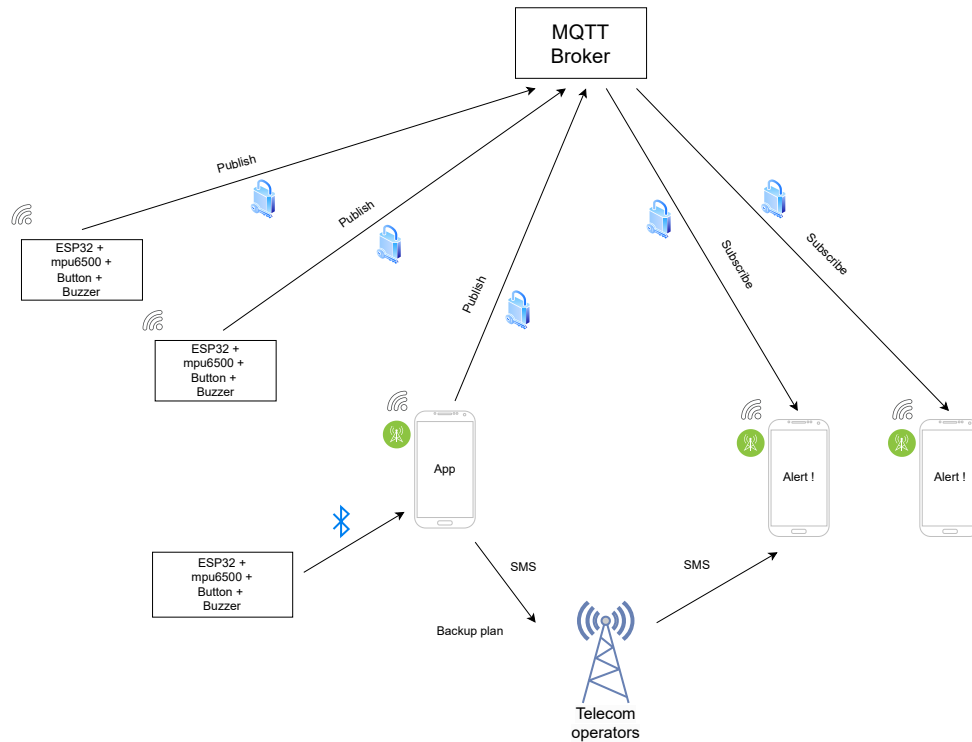


Figure 1: General System Design

2.2 SYSTEM FLOW CHART

The system comprises three roles: the IoT device, the sender application, and the receiver application.

The receiver application has a straightforward function: it alerts the caregiver upon receiving an alert message. To achieve this, it continuously subscribes to the MQTT alert topic. Once an alert is received, a full-screen notification is displayed, allowing the caregiver to either dismiss or acknowledge the alert. If acknowledged, a confirmation message is sent back to indicate that the caregiver is monitoring the situation and will take the necessary action.

Figure 2 outlines the process triggered when the mobile phone receives an alert from the IoT device via Bluetooth. Upon receiving the alert, the phone first attempts to transmit the alert using MQTT. It then subscribes to the corresponding topic to await an acknowledgement. If the message is not successfully delivered after 10 attempts, the application switches to a backup method by sending an SMS message.

Figure 3 illustrates the IoT device's operational logic, which transitions among three modes: sleep, analyze, and transmit. In sleep mode, the device remains inactive when no fall is detected. When the accelerometer registers a sudden movement, an external interrupt wakes up the IoT device, which then analyzes the accelerometer data to determine whether a fall has occurred. If no fall is detected, the device returns to sleep mode. However, if a fall is confirmed, the

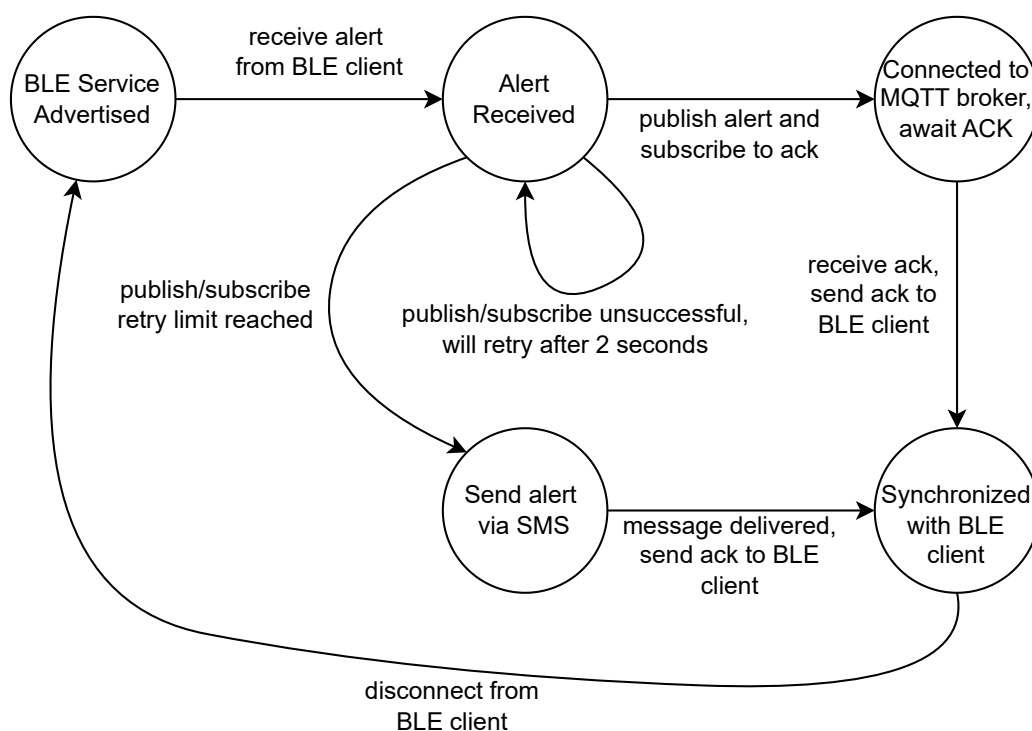


Figure 2: Flow Chart of Sender Application

device triggers an audible alert via a buzzer, providing the user with a 10-second window to cancel the alert. If the alert is not canceled within this period, the system assumes that the user is in danger and requires assistance, such as emergency medical services. Depending on its configuration, the IoT device will attempt to communicate exclusively via Bluetooth or Wi-Fi, or use both methods for redundancy. Receipt of an acknowledgement via either communication channel indicates that the caregiver has successfully received the alert, prompting the IoT device to revert to sleep mode. Conversely, if no acknowledgement is received within the specified time frame, the buzzer plays an alternative melody to signal that the user requires immediate attention from nearby individuals.

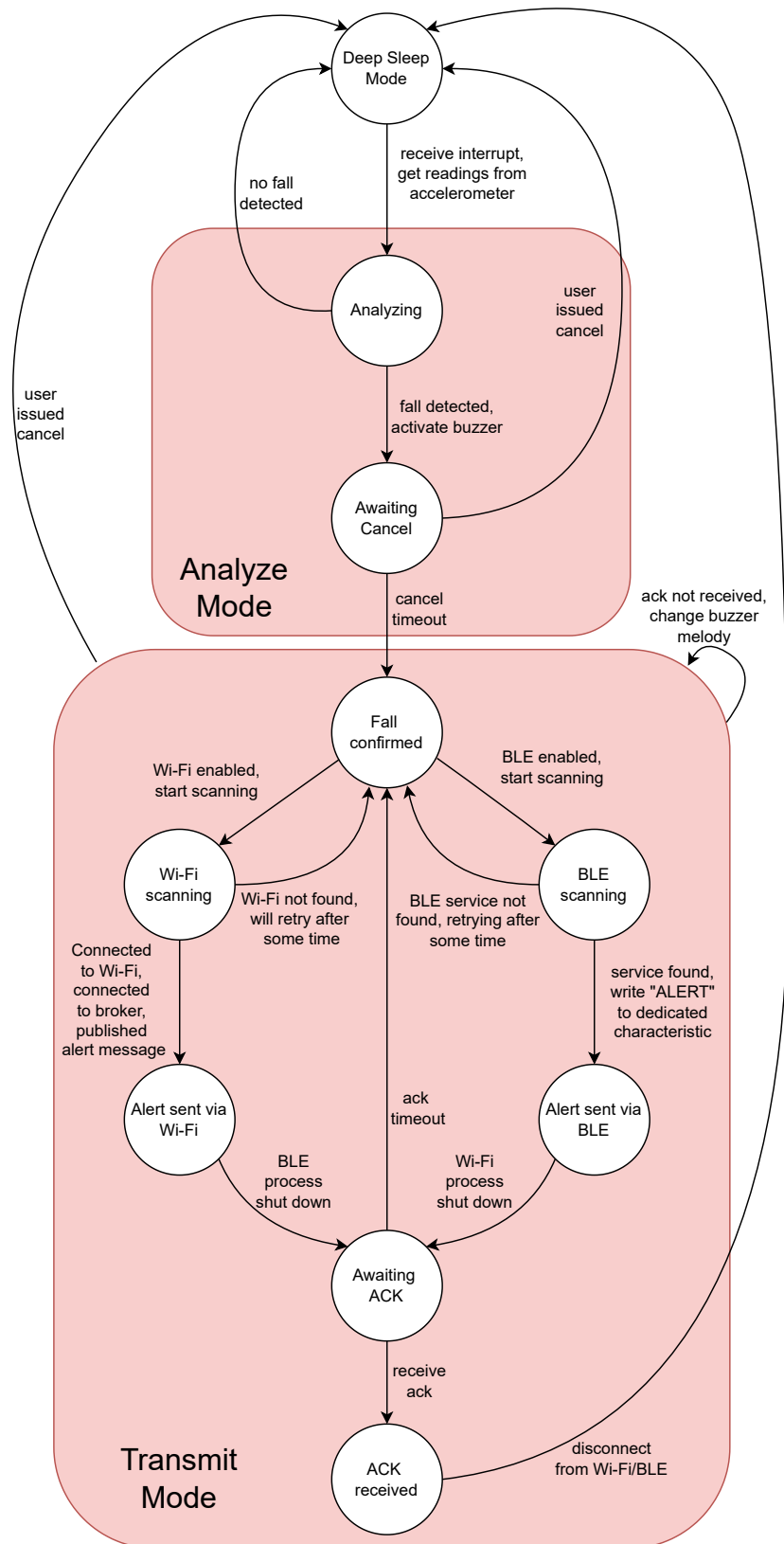


Figure 3: Flow Chart of Device

2.3 APPLICATION DESIGN

The mobile application allows users to manage configurations, which are divided into two categories: “attached” configurations for the sender side of alert delivery and “subscribed” configurations for the receiver side of alert communication. Figure 4 shows the main page, which lists current configurations. A floating plus button at the lower right corner provides a means to add new configurations. Users can subscribe to alert messages by entering the corresponding IoT device ID, or add new attached devices by scanning and configuring them.

When configuring the IoT device, the application initiates a scan for available devices and presents them on a dedicated scanning page. After establishing a connection with the IoT device, users can input Wi-Fi credentials, can enable/disable the SMS backup delivery method, and enable/disable location sharing (Figure 5). The application also offers the functionality to test Wi-Fi credentials before they are saved to the IoT device. Once configurations are successfully added, user is redirected to the main page.

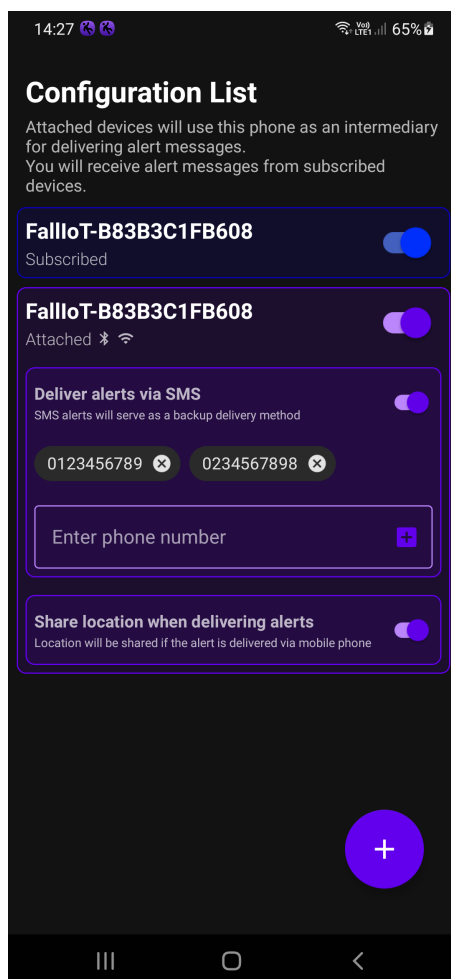


Figure 4: Main page

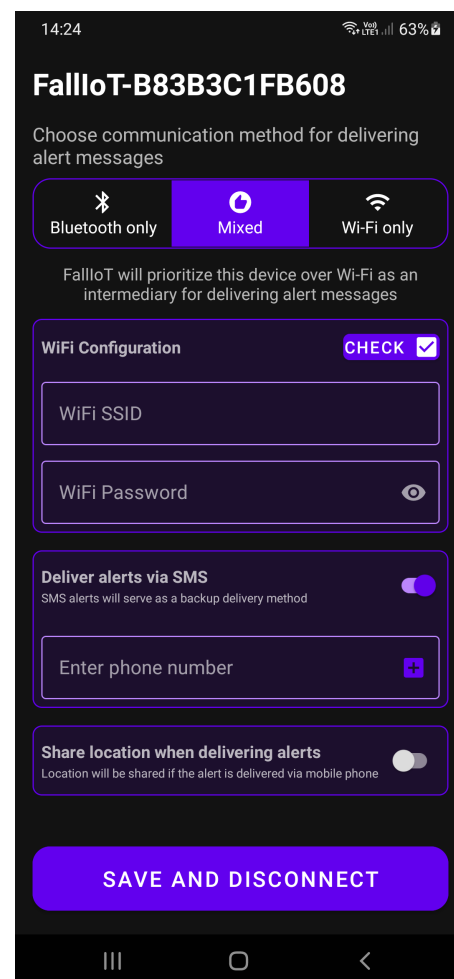


Figure 5: Configuration page

3

HARDWARE DESIGN

3.1 CIRCUIT AND PROTOTYPE

The prototype board on Figure 6 comprises two primary modules: the ESP32 devkit v1 and the MPU6500. The ESP32 serves as a microcontroller with integrated Bluetooth and Wi-Fi communication capabilities, while the MPU6500 functions as an accelerometer sensor to detect falls. When a free fall is detected, the MPU6500 issues a wake-up interrupt to the ESP32, which then initiates the process of analyzing the event and transmitting an alert message. An integrated buzzer provides an audible indication of the device's status during alert transmission, and a dedicated button allows for either canceling the alert or switching the device into configuration mode. At the time of writing this report, the prototype lacks a battery; however, incorporating an independent power source is essential for the final design. A Li-ion 3.7V, 600 mAh rechargeable battery is proposed for this purpose.

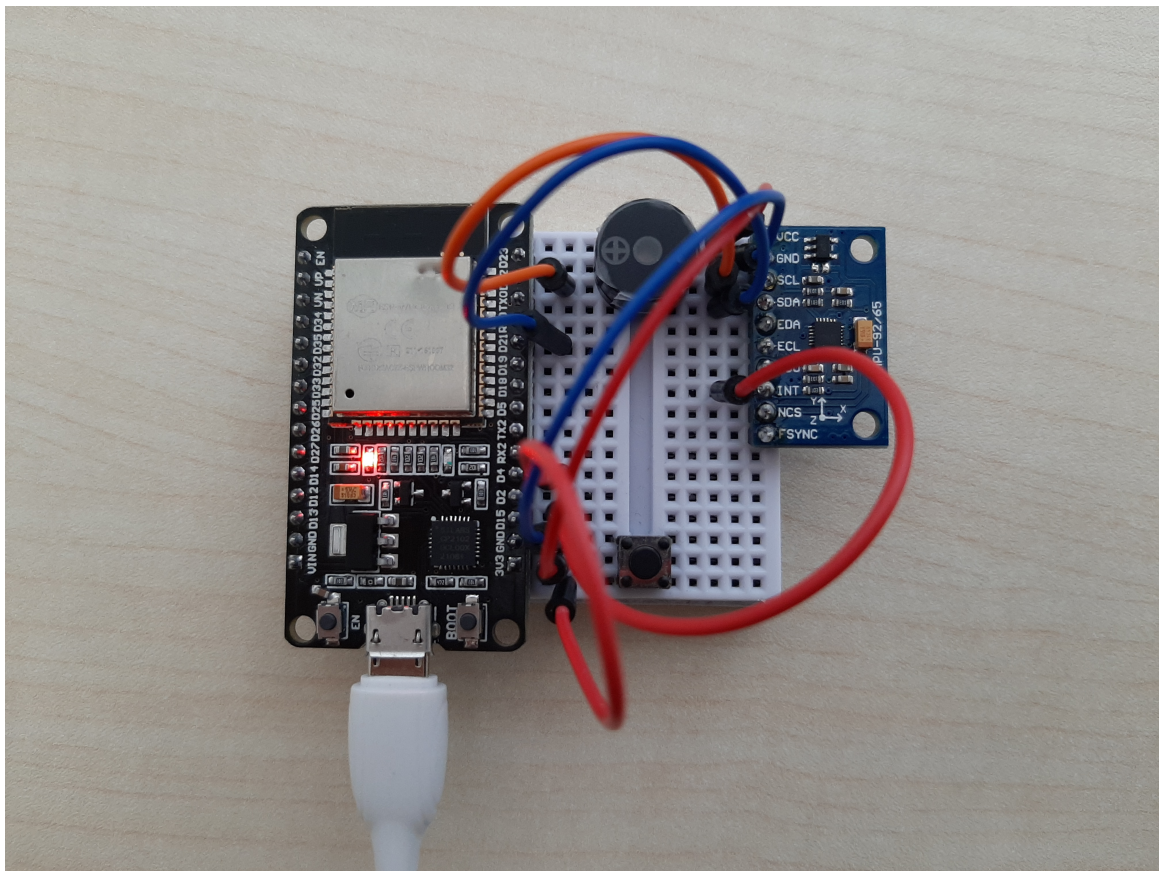


Figure 6: Prototype

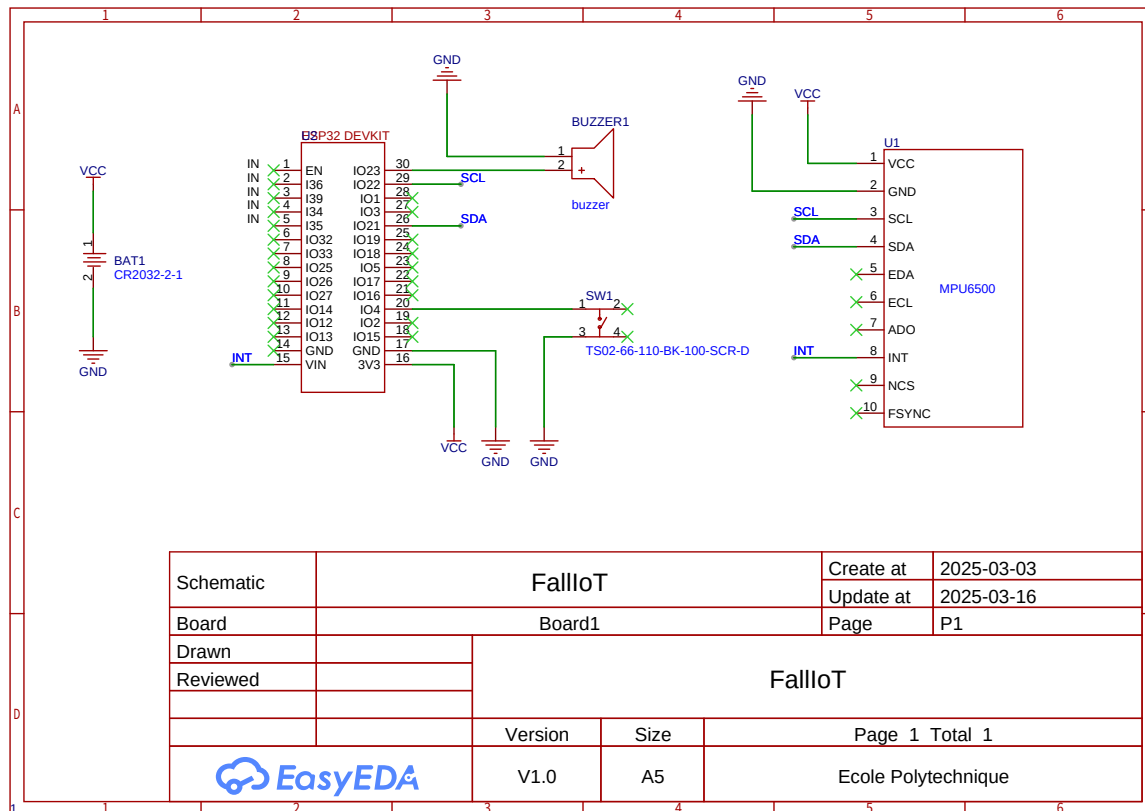


Figure 7: Circuit Schematic

3.2 POWER CONSUMPTION AND BATTERY LIFE

The device operates in three distinct modes: sleep, detect, and transmit. When the user is sleeping, sitting, or walking at a slow pace, the device remains in sleep mode, where power consumption is minimal.

Operating Mode	Current
ESP32 - Deep Sleep Mode	10 μ A
ESP32 – Active Mode	<240mA
MPU6500 - Low Power Mode	18.65 μ A
MPU6500 - Operating Mode	3.4mA

Table 1: Power consumption in different operating modes

During activities such as sports or other vigorous movements, it switches to detect mode. The device enters transmit mode only when a fall is detected; in this mode, the user can manually disable if an alert was false positive.

Operating Mode	Current	Battery Life
Sleep Mode	28.65 μ A	1340 days
Detect Mode	3.41mA	176h (more than 7 days)
Transmit Mode	243mA	2.47h

Table 2: Power consumption in different operating modes

The battery is designed to support daily use for an estimated period of two weeks to one month, minimizing the need for frequent recharging.

Job	Sitting and Sleep Time (Sleep Mode)	Battery Life
Blue-Collar Workers	10~14h	12.5d~17.6d
White-Collar Workers	14~18h	17.6d~29.3d
Students	14~16h	17.6d~22d
Preschoolers (children between 3~5 years old)	14~18h	17.6d~29.3d

Table 3: Battery Life of Device in Different Jobs

3.3 COST

Component	Price
ESP32	3.1€
MPU6500	0.7€
Buzzer	0.9€
Battery	2.8€
Other	~1€
Total	~9€

Table 4: Cost

4

DESIGN CHOICES

Throughout the project development, multiple design alternatives were evaluated, each with its own set of trade-offs. The designs described above were identified as the most pertinent for addressing the project's objectives.

For microcontroller selection, the ESP32 was determined to be the optimal solution when balancing cost and functionality, particularly due to its integrated Wi-Fi and Bluetooth capabilities. Although the ESP8266 is less expensive, its lack of built-in Bluetooth would necessitate the addition of an external Bluetooth module, thereby increasing the overall cost. Consequently, the ESP32 was chosen as the final option.

The microcontroller utilizes Bluetooth Low Energy rather than Bluetooth Classic, as the data exchanged between the IoT device and the smartphone is minimal. This choice contributes to a reduction in power consumption for both devices.

The IoT device is configurable to use Bluetooth only, Wi-Fi only, or both methods concurrently for delivering alert messages. This flexibility allows users to tailor the system to their specific needs. For instance, individuals who predominantly remain at home with Wi-Fi access, or who do not possess a smartphone, may prefer a Wi-Fi-only configuration, while others may opt for Bluetooth when Wi-Fi connectivity is not available. Although employing a single method reduces power consumption and extends battery life, the use of both methods ensures the reliability of critical message delivery.

The motivation for choosing MQTT broker as an intermediary to communicate alert messages is the advantage of isolation of subscribers and publishers as it is not required to know who exactly is subscribing to alert messages. Also, centralized nature of uniting publishers and subscribers makes it easier to develop communication semantics on top MQTT protocol. Also, because the transmitting information is only text, WebRTC and RTSP are not suitable in the design. Though, not yet implemented in the project, MQTT allows secure communication, which is important to be considered in final design.

The project leverages the widespread availability of smartphones to streamline the delivery of alert messages and reduce overall system costs. The smartphone provides the advantage of sending SMS messages, providing a backup method to deliver critical alerts. It also facilitates the sharing of location information, which is essential for caregivers responding to an alert.

5

SOFTWARE IMPLEMENTATION DETAILS

This section details the software implementation for the IoT device, mobile application, and MQTT broker. All source code is accessible at: gitlab.com/fallert. Fallert is the name of the project and at the same time is the name of the mobile application and its associated code. FallIoT refers to the IoT device and its related code. FallBridge refers to code associated with MQTT broker configuration.

Software for the ESP32 was developed using PlatformIO with the Arduino framework. The ESP32 predominantly operates in deep sleep mode and is awakened by external interrupts—one triggered by a button and another by the accelerometer. When in sleep mode, a 3–4 second button press transitions the microcontroller into configuration mode. In this state, the ESP32 initiates a Bluetooth Low Energy (BLE) server with a defined timeout. If no connection is established before the timeout, the ESP32 returns to deep sleep; otherwise, it exchanges data with the connected device. Specifically, the ESP32 advertises a BLE service with a UUID common to all FallIoT devices (the same UUID is used in the mobile application to filter BLE services during scanning). This service contains characteristics for operation mode, Wi-Fi SSID, Wi-Fi password, Wi-Fi testing, and phone identifier. Data is read from these characteristics from the preferences memory, and key parameters—including the phone identifier, operation mode, and Wi-Fi credentials—are permanently stored in the ESP32's flash memory via the preferences library, which uses the default NVS partition. When a mobile phone connects to a ESP32 BLE server, it reads the characteristics related to Wi-Fi credentials and operation mode. Upon completing the configuration process, the mobile phone sends a submission command, prompting the ESP32 to update the characteristics and store the new settings in preferences. The phone identifier characteristic holds the mobile application's unique UUID, which is used later by the FallIoT device during BLE service scans after detecting a fall. The Wi-Fi test characteristic communicates binary information to verify the entered credentials. Once the mobile phone disconnects, the ESP32 deactivates the BLE server and returns to deep sleep mode.

Another wake-up source is an interrupt from the MPU6500 accelerometer. Communication with this sensor occurs via the I2C protocol using the `libdriver/mpu6500` library. The MPU6500 issues an interrupt when it detects a free fall along the vertical axis, waking the ESP32 from deep sleep. The ESP32 then reads the accelerometer's Z-axis data and measures the duration of the free fall, ending with a sharp impact interrupt. It compares the distance traveled during the fall to an average human chest-to-ground height. If the measurement is consistent with a realistic fall, the alert process is initiated; otherwise, the microcontroller resumes sleep mode.

When a fall is confirmed, the ESP32 sends an alert message based on its preconfigured operational mode. Implementation wise, exclusive Wi-Fi and Bluetooth communication methods are subset of mixed operational mode, in which both of these methods can be used to communicate alert message; therefore, the mixed operational mode will be discussed here. Upon detecting a fall, the ESP32 submits two tasks to different cores: one for Bluetooth and one for

Wi-Fi. Since both peripherals share the same radio circuitry, a mutex is used to ensure that only one type of scanning (either Bluetooth or Wi-Fi) occurs at a time. Bluetooth is given priority due to its additional functionalities, such as offering a backup communication method and facilitating location sharing. Once communication is established via either channel, the alternate task is terminated to conserve energy. In this state, the FallIoT device sends the alert message to the recipient—either directly to the MQTT broker (via Wi-Fi) or to the phone (via Bluetooth)—and waits for an acknowledgement. If an acknowledgement is not received within a predetermined timeframe, the device restarts the BLE and Wi-Fi scanning processes. Conversely, if an acknowledgement is received, the FallIoT device returns to deep sleep.

FallBridge refers to the container hosting the MQTT broker deployed in the cloud. The broker configuration is basic, permitting anonymous connections. The cloud service provider (DigitalOcean) assigns a public IPv4 address to the droplet, and this IP address, along with port 1883, is used for connectivity from both the mobile application and the FallIoT device.

The mobile application, developed for Android (API levels 24 and above, tested up to API 35), facilitates configuration of FallIoT devices and subscription to their alert messages. Alert message delivery and reception is managed by two separate foreground services. Running these as foreground services is necessary to prevent newer versions of Android from terminating background processes, which could otherwise disable the BLE server for attached FallIoT devices or disrupt continuous network connectivity for subscribed devices. If at least one configuration is enabled for attached devices, the BLE service starts; similarly, the MQTT subscription service starts if at least one configuration is enabled for subscribed devices. The `hivemq` library is utilized for MQTT publish-subscribe functionality. Once the MQTT service starts, it retrieves enabled configurations from preferences and subscribes to the corresponding topics. Upon receiving an alert message, the application rings, vibrates, and displays a full-screen notification with options to either discard or acknowledge the alert. Acknowledgement is then published to the corresponding topic.

When the BLE service is initiated, the application starts a BLE server and advertises a service with a one-time generated UUID that remains unique unless the application is reinstalled. The service advertises characteristics for each attached FallIoT device. To ensure each device has a dedicated characteristic for alert communication, the application replaces the last 12 characters of the one-time generated UUID (used as the BLE service ID) with the FallIoT identifier, advertising the resultant UUIDs as characteristics. Consequently, when a FallIoT device connects to this service, it locates the characteristic whose ending matches its identifier and uses that characteristic for message exchange. When the application detects an update to a characteristic, it determines the corresponding FallIoT device by reading the ending of the characteristic. After transmitting an alert message from the FallIoT device, the application attempts to forward it to the MQTT broker while simultaneously subscribing to the corresponding topic to await an acknowledgement. If the acknowledgement is successfully received, the application writes back to the corresponding characteristic and notifies the connected devices.

6

FUTURE CONSIDERATIONS

The project is currently partially completed and lacks several key functionalities. For instance, the fall detection algorithm is simplified—it only considers the vertical axis, lacks a calibration mechanism to compensate for g-force, and does not incorporate gyroscope data. As a result, it cannot precisely track the device’s position or offer a detailed analysis. Moreover, the MPU6500 sensor is outdated and has been deprecated by its manufacturer, suggesting that newer alternatives should be explored.

Regarding MQTT communications, neither authentication nor data encryption has been implemented at this stage; these security measures should be integrated in future iterations.

Additionally, there is a security concern related to the BLE server. The mobile application starts the BLE server and advertises the service with its characteristics. When the FallIoT device scans for available services, it merely verifies the presence of the expected service and characteristic and then performs read/write operations without any identity verification. This lack of authentication renders the system vulnerable to false alerts; any device that discovers the BLE service and writes an “ALERT” string to one of the characteristics could inadvertently trigger an alert event on the mobile phone.

REFERENCES

- [1] World Health Organization. *Falls*. Retrieved from <https://www.who.int/news-room/fact-sheets/detail/falls>
- [2] Camp, K., Murphy, S., & Pate, B. (2024). Integrating Fall Prevention Strategies into EMS Services to Reduce Falls and Associated Healthcare Costs for Older Adults. *Clinical Interventions in Aging*, 19, 561–569. <https://doi.org/10.2147/cia.s453961>
- [3] Apple Inc. *Use fall detection with Apple Watch*. Retrieved from <https://support.apple.com/en-us/108896>
- [4] Apple Inc. *Apple Watch Battery*. Retrieved from <https://www.apple.com/watch/battery/>
- [5] Apple Inc. *Apple Watch*. Retrieved from <https://www.apple.com/watch/>
- [6] Magdy, K. *ESP32 Sleep Modes & Power Consumption in Each Mode*. Retrieved from <https://deepbluembedded.com/esp32-sleep-modes-power-consumption/>
- [7] TDK InvenSense. *MPU-6500 Product Specification*. Retrieved from <https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6500/>
- [8] Shenzhen Yishenglong Technology Co., Ltd. *ESP32 Development Board Wireless WiFi+Bluetooth 2-in-1 Dual Core CPU Low Power ESP-32 Control Board ESP-32S*. Retrieved from https://www.alibaba.com/product-detail/p_1601295401806.html
- [9] High-quality GY-6500 6DOF 6-axis Attitude Sensor Module. Retrieved from <https://www.alibaba.com>
- [10] Temu. *Kuiper UN9 Smartwatch*. Retrieved from <https://www temu.com>
- [11] YDL Battery. *3.7V 600mAh 503040 Lithium Polymer Ion Battery*. Retrieved from <https://ydlbattery.com/fr-fr/products/3-7v-600mah-503040-lithium-polymer-ion-battery>
- [12] ESP32 Pinout Reference: Which GPIO pins should you use? <https://randomnerdtutorials.com/esp32-pinout-reference-gpios/>